

Sampled-Ability Mathematics for 4PL IRT with J Scales

Scope

This note describes the sampled-ability optimization scheme for a multidimensional 4PL IRT model with J latent scales. It focuses on the mathematical steps:

- person posterior approximation,
- sigma-point construction and weights,
- weighted item-parameter optimization objective/gradient,
- alternating person/item updates.

The goal is to approximate uncertainty-aware item updates without full MCMC.

1) Model Setup (Row-Level 4PL)

For observation row r , let:

- person index: $i(r)$,
- binary response: $y_r \in \{0, 1\}$,
- latent ability vector for person i : $\theta_i \in \mathbb{R}^J$,
- effective loading vector: $a_r \in \mathbb{R}^J$,
- effective difficulty: b_r ,
- lower asymptote: c_r ,
- upper asymptote: d_r , with $0 \leq c_r < d_r \leq 1$.

Linear predictor:

$$\eta_r = a_r^\top \theta_{i(r)} - b_r.$$

Logistic core:

$$g_r = \sigma(\eta_r) = \frac{1}{1 + e^{-\eta_r}}.$$

4PL success probability:

$$p_r = c_r + (d_r - c_r)g_r.$$

Row log-likelihood contribution:

$$\ell_r = y_r \log p_r + (1 - y_r) \log(1 - p_r).$$

2) Person Prior and Local Gaussian Posterior

Assume a Gaussian prior on ability:

$$\theta_i \sim \mathcal{N}(\mu_0, \Sigma_0), \quad Q_0 = \Sigma_0^{-1}.$$

In sampled-ability, each person gets a local Gaussian approximation:

$$q_i(\theta_i) \approx \mathcal{N}(\hat{\theta}_i, \Sigma_i).$$

The approximation uses a precision update:

$$Q_i = Q_0 + \sum_{r:i(r)=i} w_r a_r a_r^\top.$$

Then:

$$\Sigma_i = Q_i^{-1}.$$

In the current expected-information implementation, row weight is:

$$w_r = \frac{\left(\frac{\partial p_r}{\partial \eta_r}\right)^2}{p_r(1-p_r)}, \quad \frac{\partial p_r}{\partial \eta_r} = (d_r - c_r)g_r(1 - g_r).$$

So:

$$w_r = \frac{(d_r - c_r)^2 g_r^2 (1 - g_r)^2}{p_r(1 - p_r)}.$$

This is evaluated at current $\hat{\theta}_i$. A diagonal jitter is added as needed for stable Cholesky/inversion.

3) Sigma-Point Construction per Person

Given $(\hat{\theta}_i, \Sigma_i)$, construct $2J + 1$ deterministic sigma points.

Let $\lambda > 0$ (here fixed to 1 in the implementation) and

$$s = \text{sigmaScale} \cdot \sqrt{J + \lambda}.$$

Compute an upper-triangular square root U_i such that

$$U_i^\top U_i \approx \Sigma_i$$

after stabilization.

Sigma points:

$$\begin{aligned} \theta_i^{(0)} &= \hat{\theta}_i, \\ \theta_i^{(k)} &= \hat{\theta}_i + s U_{i,\cdot k}, \quad k = 1, \dots, J, \\ \theta_i^{(J+k)} &= \hat{\theta}_i - s U_{i,\cdot k}, \quad k = 1, \dots, J. \end{aligned}$$

Shared sigma weights:

$$\omega_0 = \frac{\lambda}{J + \lambda}, \quad \omega_s = \frac{1}{2(J + \lambda)} \text{ for each non-central point.}$$

These satisfy:

$$\sum_{s=0}^{2J} \omega_s = 1.$$

4) Building Global Ability Templates

For each sigma index $s \in \{0, \dots, 2J\}$, create a full ability template for all persons:

$$\Theta^{(s)} = [\theta_1^{(s)}, \dots, \theta_N^{(s)}]^\top.$$

Each $\Theta^{(s)}$ is inserted into the unconstrained parameter vector only at ability coordinates, while item coordinates remain free in the item step.

Conceptually this yields $2J + 1$ fixed-ability worlds.

5) Item-Parameter Step with Weighted Multi-Evaluation

Let ϕ be item-side free parameters in the current block update. For each sigma template s , define objective:

$$\mathcal{L}^{(s)}(\phi) = \log p(y, \phi, \Theta^{(s)}).$$

Sampled-ability item objective is the weighted mixture:

$$\bar{\mathcal{L}}(\phi) = \sum_{s=0}^{2J} \omega_s \mathcal{L}^{(s)}(\phi).$$

Gradient used by optimizer:

$$\nabla_{\phi} \bar{\mathcal{L}}(\phi) = \sum_{s=0}^{2J} \omega_s \nabla_{\phi} \mathcal{L}^{(s)}(\phi).$$

In practice, each optimizer evaluation runs all sigma-template evaluations and returns the weighted sum objective + weighted gradient.

6) Person-Parameter Step

With item parameters fixed, optimize person-side free parameters (point estimate step) against the standard objective:

$$\max_{\psi} \log p(y, \psi, \phi),$$

where ψ denotes person-side free coordinates.

After this person step, recompute local Gaussian quantities and sigma points for the next item step.

7) Alternating Outer Loop

For outer iteration $t = 1, \dots, T$:

1. **Person update (point estimate)**: optimize person block.
2. **Posterior/sigma refresh**: compute $Q_i, \Sigma_i, \theta_i^{(s)}, \omega_s$.
3. **Item update (uncertainty-aware)**: optimize item block using $\bar{\mathcal{L}}(\phi)$.

A combined gradient norm from person and item substeps is used as an outer-loop convergence signal.

8) Why Weighting Works Here

The item step approximates

$$\mathbb{E}_{q(\Theta)}[\log p(y, \phi, \Theta)]$$

with a deterministic quadrature rule based on sigma points:

$$\mathbb{E}_{q(\Theta)}[\cdot] \approx \sum_s \omega_s(\cdot) \big|_{\Theta^{(s)}}.$$

This captures local uncertainty in abilities while keeping a deterministic, gradient-based optimization routine.

9) Notes on J -Scale Behavior

- Number of sigma templates per outer iteration is $2J + 1$.
- Item-step optimizer cost scales roughly linearly in $2J + 1$, because each target/gradient evaluation aggregates all templates.
- For $J = 1$: 3 templates; for $J = 3$: 7 templates; etc.

This is usually far cheaper than Monte Carlo sampling over abilities while still encoding second-order local uncertainty structure.